

■ 3.9.4 Numerical Evaluation of Sums and Products

<code>NSum[f, {i, imin, imax}]</code>	find a numerical approximation to the sum $\sum_{i=imin}^{imax} f$
<code>NSum[f, {i, imin, imax, di}]</code>	use step di in the sum
<code>NProduct[f, {i, imin, imax}]</code>	find a numerical approximation to the product $\prod_{i=imin}^{imax} f$

Numerical sums and products.

This gives a numerical approximation to $\sum_{i=1}^{\infty} \frac{1}{i^3}$.

```
In[1]:= NSum[1/i^3, {i, 1, Infinity}]
Out[1]= 1.20206
```

Mathematica does not know the exact result for this sum, so leaves it in symbolic form.

```
In[2]:= Sum[1/i^3, {i, 1, Infinity}]
Out[2]= Sum[i^-3, {i, 1, Infinity}]
```

You can apply `N` explicitly to get a numerical result.

```
In[3]:= N[%]
Out[3]= 1.20206
```

The way `NSum` works is to include a certain number of terms explicitly, and then to try and estimate the contribution of the remaining ones. There are two approaches to estimating this contribution. The first uses the Euler-Maclaurin method, and is based on approximating the sum by an integral. The second method, known as Wynn's epsilon method, samples a number of additional terms in the sum, and then tries to fit them to a polynomial multiplied by a decaying exponential.

<code>WorkingPrecision</code>	<code>Precision[1.]</code>	number of digits of precision to use in internal computations
<code>Method</code>	<code>Automatic</code>	<code>Integrate</code> (Euler-Maclaurin method) or <code>Fit</code> (Wynn epsilon method)
<code>Terms</code>	15	number of terms to include explicitly
<code>ExtraTerms</code>	11	number of terms to use for extrapolation in the Wynn epsilon method
<code>AccuracyGoal</code>	6	number of digits of accuracy to try and get in the Euler-Maclaurin method

Options for `NSum`.

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; <http://www.wolfram.com/>; 1-800-441-6284.

© 1988 Wolfram Research, Inc. Permission is hereby granted for web users to make one paper copy of this page for their personal use. Further reproduction, or any copying of machine-readable files (including this one) to any server computer, is strictly prohibited.

If you do not explicitly specify the method to use, **NSum** will try to choose between the methods it knows. In any case, some implicit assumptions about the functions you are summing have to be made. If these assumptions are not correct, you may get inaccurate answers.

The most common place to use **NSum** is in evaluating sums with infinite limits. You can, however, also use it for sums with finite limits. By making implicit assumptions about the objects you are evaluating, **NSum** can often avoid doing as many function evaluations as an explicit **Sum** computation would require.

This finds the numerical value of $\sum_{n=0}^{100} e^{-n}$ by extrapolation techniques.

```
In[4]:= NSum[Exp[-n], {n, 0, 100}]
Out[4]= 1.58198
```

You can also get the result, albeit much less efficiently, by constructing the symbolic form of the sum, then evaluating it numerically.

```
In[5]:= Sum[Exp[-n], {n, 0, 100}] //N
Out[5]= 1.58198
```

NProduct works in essentially the same way as **NSum**, except that only the Wynn epsilon method can be used: there is no way to approximate a product directly by an integral. (If necessary, you can nevertheless form a product from an exponential of a sum of logarithms.)